

IRI Resource Architecture

Types, type-specific attributes, hypermedia links, and agentic discovery

High-level overview of the DOE Integrated Research Infrastructure Facility API model



John MacAuley • August 2026

DOE Integrated Research Infrastructure

Why change the Resource architecture

IRI spans heterogeneous facilities, technologies, services, and independently operated API deployments.

PROBLEM

Before: clients infer too much

- A common Resource schema cannot contain every compute, storage, network, transfer, and inference-specific property.
- Clients need out-of-band knowledge to understand which fields apply to which kinds of resources.
- Relationships and operation endpoints are often implied by URI conventions or SDK logic.
- AI/MCP agents are tempted to guess API paths from resource identifiers and prior examples.

Result: brittle clients + facility-specific coupling

After: resources advertise semantics

resource_type

What kind of Resource is this?

attributes

What type-specific data applies?

_links

What is related or actionable here?

OpenAPI

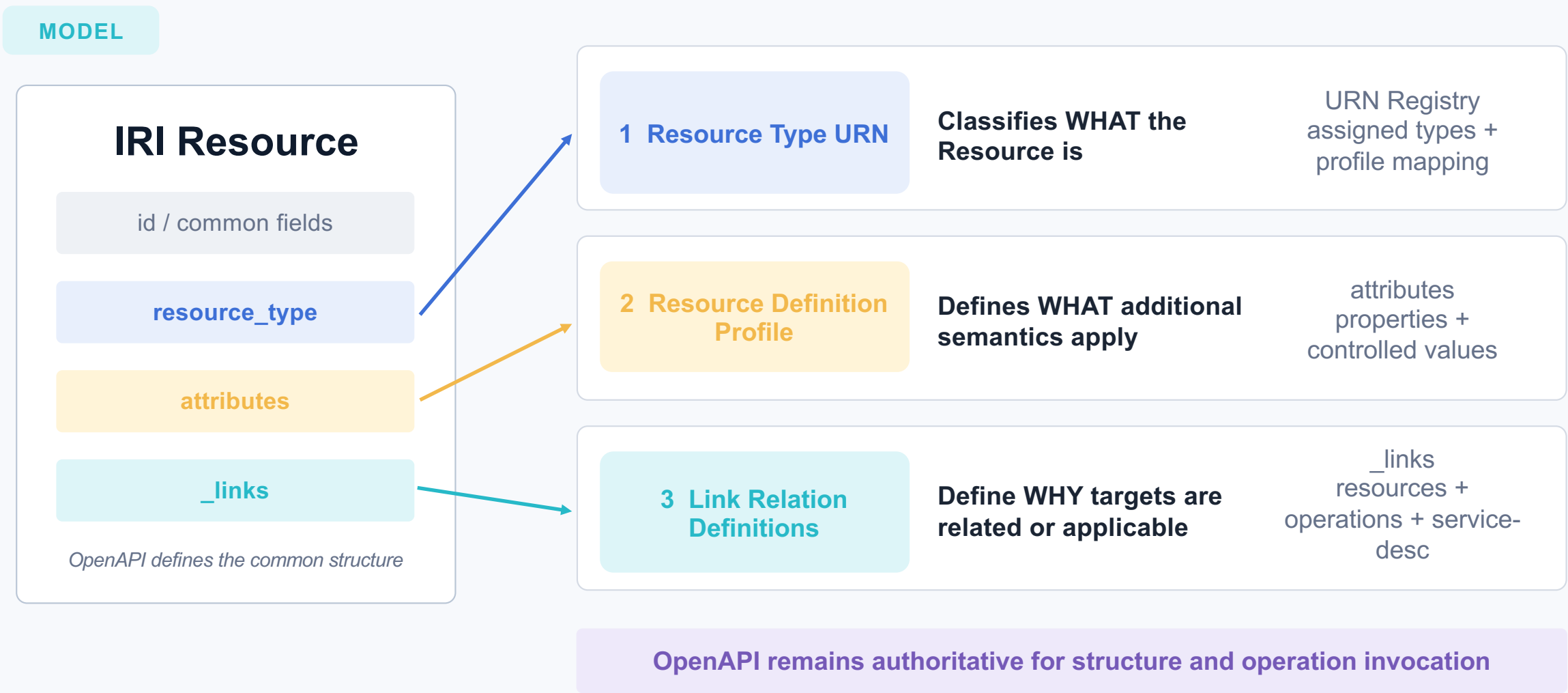
How is an operation invoked?

Result: discoverable semantics + less path speculation

Sources: [rfc/rfc-type-specific-attributes.md](#); [rfc/rfc-hal-links.md](#); [registry/profiles/status/resource.md](#)

The architecture at a glance

One Resource representation; multiple semantic layers with clear ownership.



Source: [registry/README.md](#); [rfc/rfc-type-specific-attributes.md](#); [rfc/rfc-hal-links.md](#)

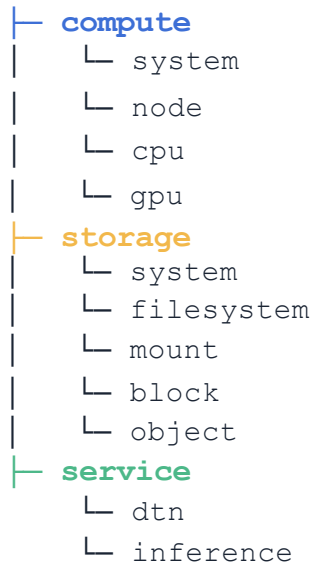
`resource\_type`: stable classification without OpenAPI churn

The URN registry owns the taxonomy; OpenAPI only needs an extensible string.

RESOURCE TYPE

Semantic taxonomy

`urn:doe-iri:resource`



Classification hierarchy ≠ topology

Compute system, node, CPU,
and GPU remain sibling
resource types.
Containment is expressed with
links.

What the registry provides

Assigned URN	e.g. urn:doe-iri:resource:compute:system
Semantic definition	Managed computing environment
Parent	urn:doe-iri:resource:compute
Lifecycle	active / provisional / deprecated
Profile mapping	compute/system Resource Definition Profile

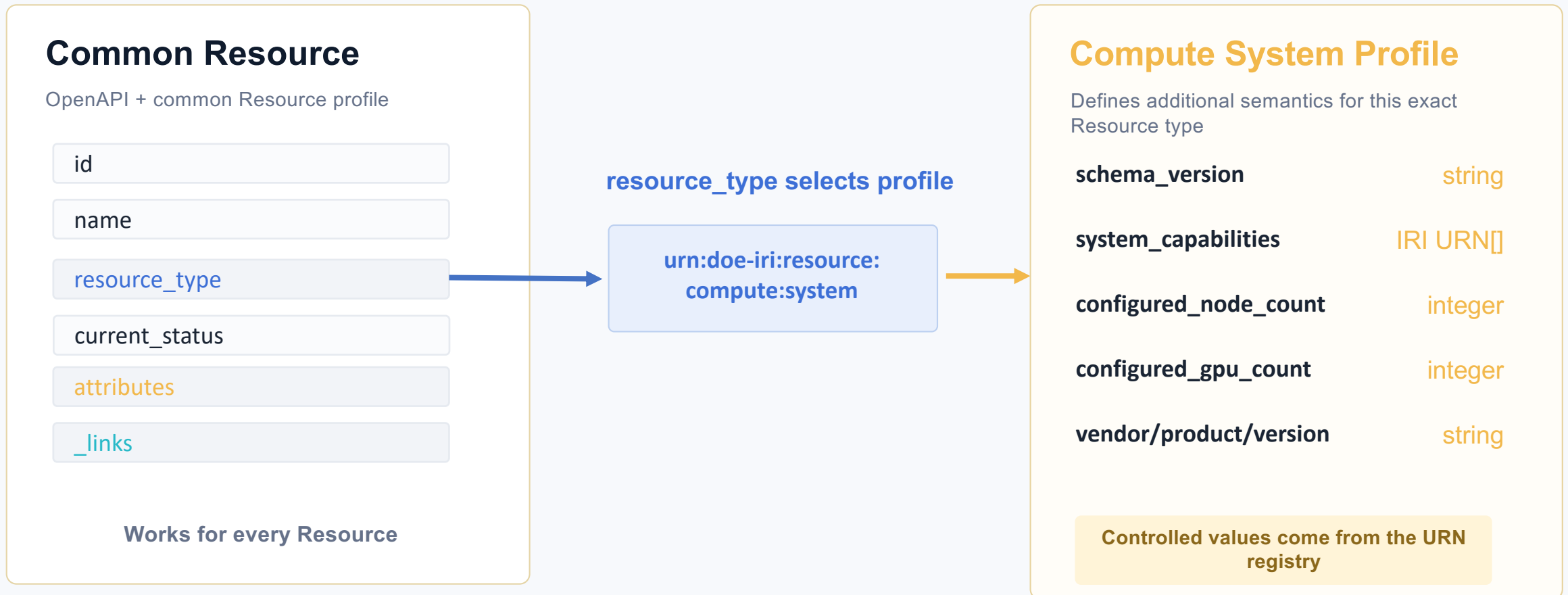
**New resource types can be registered without
changing the common Resource schema.**

Source: [registry/urns/resource-types.md](#); [registry/profiles/status/resource.md](#)

`attributes`: type-specific data governed by profiles

Profiles add semantics without turning the common Resource schema into a union of every technology.

PROFILES



Sources: [rfc/rfc-type-specific-attributes.md](#); [registry/profiles/resource-definition/compute/system.md](#); [registry/urns/attributes.md](#)

`\_links`: explicit topology, navigation, and operation affordances

Relations say why a target matters; Link Object metadata says where and what the target is.

HYPERMEDIA

Four distinct roles

relation

WHY the target is linked

href

WHERE the target is

type

HOW the target is represented

profile

WHAT semantic profile applies to the target

Example: `iri:has-mount` → `mount Resource profile`

What each relation definition documents

- source and target representation types
- semantic meaning and target classification
- cardinality and omission behavior
- stability/volatility of the relationship
- authorization-sensitive visibility
- HAL representation examples

**Operation links identify applicability + entry point;
OpenAPI still defines method, body, errors, and security.**

Sources: *rfc/rfc-hal-links.md*; *registry/relations/README.md*

One Resource becomes self-describing

An illustrative compute-system representation combines classification, profile-governed attributes, and navigable links.

EXAMPLE

```
{
  "id": "pioneer-compute",
  "resource_type": "urn:doe-iri:resource:compute:system",
  "attributes": {
    "schema_version": "1.0.0",
    "system_capabilities": [
      "urn:doe-iri:compute:system-capability:batch-scheduling",
      "urn:doe-iri:compute:system-capability:accelerator-support"
    ],
    "configured_gpu_count": 2048
  },
  "_links": {
    "self": {
      "href": ".../status/resources/pioneer-compute",
      "profile": "https://iri.science/profiles/resource-definition/compute/system"
    },
    "iri:has-node": {
      "href": ".../status/resources/pioneer-node-001",
      "profile": "https://iri.science/profiles/resource-definition/compute/node"
    },
    "iri:submit-job": {
      "href": ".../compute/job/pioneer-compute"
    },
    "service-desc": { "href": ".../openapi.json" }
  }
}
```

resource_type

Lookup the exact Resource Type and its registered Resource Definition Profile.

attributes

Interpret the object using that profile; validate controlled values against the URN registry.

_links

Traverse only advertised relations. Relation docs explain source, target, cardinality, and meaning.

service-desc

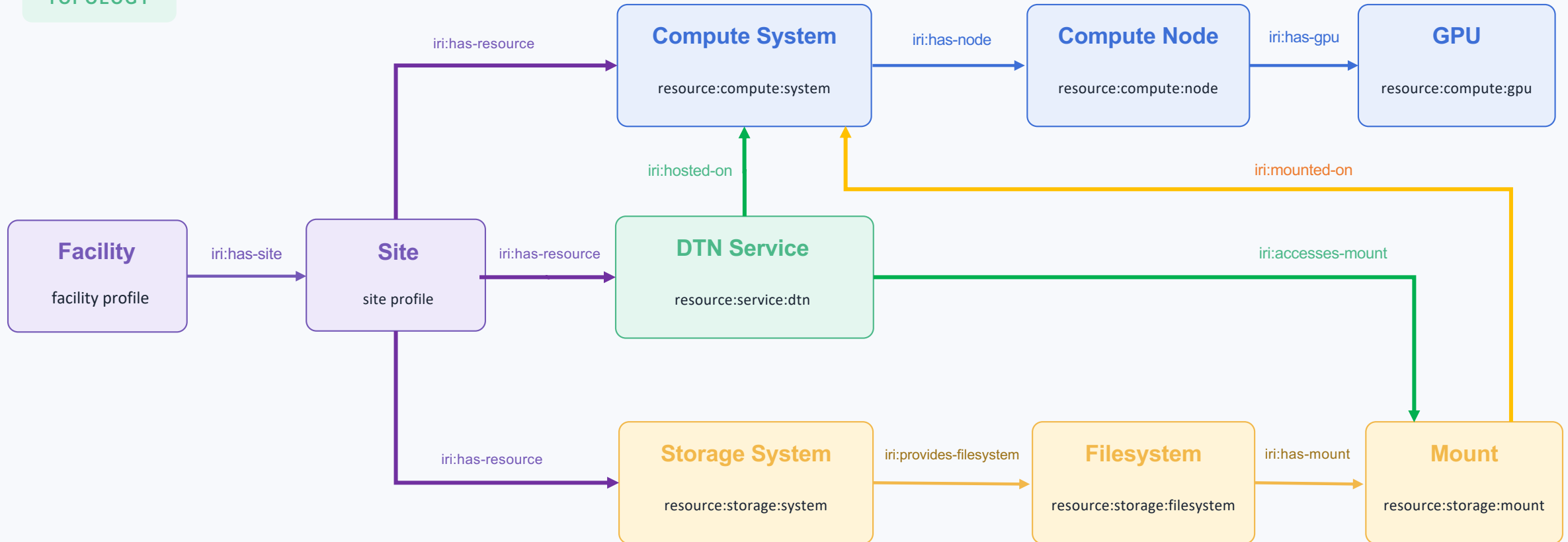
Use OpenAPI to learn invocation details instead of inferring HTTP behavior from the relation.

Illustrative composite based on [registry/examples/hpc-facility-resources.json](#) and [rfc/rfc-hal-links.md](#)

From a list of resources to a traversable facility capability graph

Relations model topology explicitly instead of encoding it in names, paths, or nested resource types.

TOPOLOGY



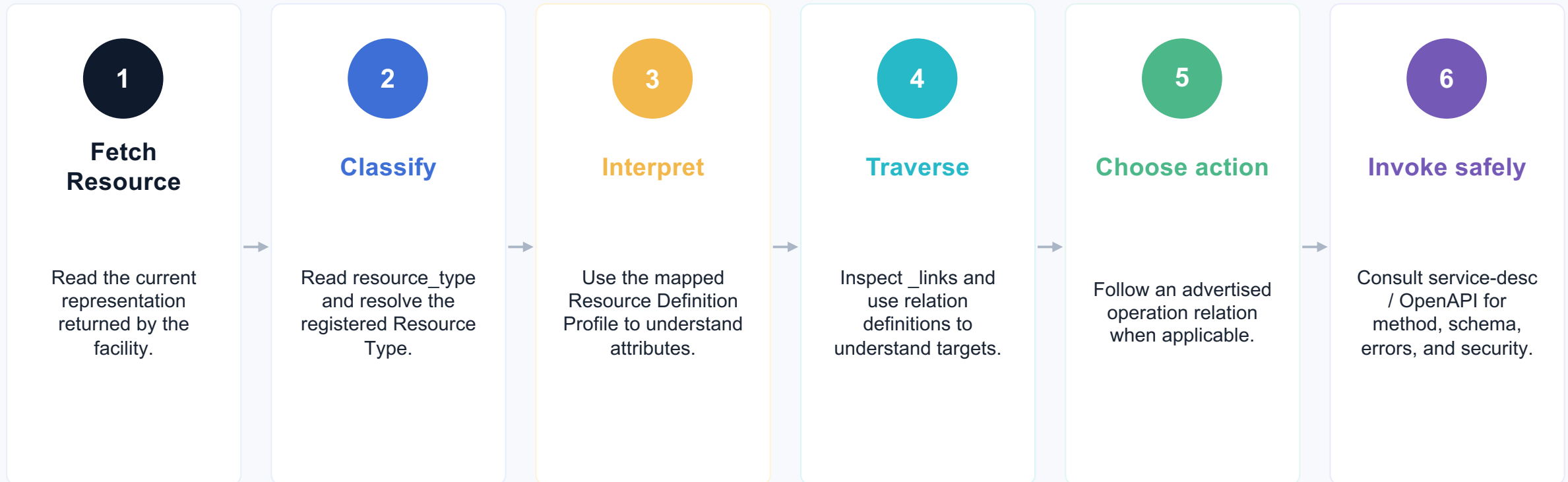
An agent can traverse only the relationships a facility chooses to expose, without needing to know the facility's internal routing structure.

Sources: [registry/relations/README.md](#); [registry/examples/hpc-facility-resources.json](#)

How an agent understands and acts on a facility Resource

The current representation becomes the grounding point; registries and profiles provide reusable semantic context.

AGENTIC WORKFLOW



Key rule: do not guess paths from resource identifiers or cached examples; use the current links and governing OpenAPI.

Source: [rfc/rfc-hal-links.md](#); [registry/profiles/status/resource.md](#)

Why split URNs, profiles, relations, and OpenAPI?

Each document class has one job, one authority boundary, and a different change cadence.

GOVERNANCE

URN Registry

What identifiers exist?

- Resource Type URNs
- Controlled attribute URNs
- Parentage + lifecycle
- Type → profile mapping

Profiles

What does a representation mean?

- Common Resource semantics
- Type-specific attributes
- Controlled value usage
- Conformance expectations

Relation Docs

Why are two things connected?

- Source + target
- Cardinality
- Target classification
- Visibility + stability

OpenAPI

How does the API work?

- JSON structure
- Operations + methods
- Request/response schemas
- Errors + security

Separation avoids duplication, keeps governance explicit, and lets each layer evolve independently without semantic ambiguity.

Source: registry/README.md and current registry organization

What this architecture buys us

A small common schema with a governed semantic extension system.

BENEFITS



Extensible taxonomy

Add a new Resource Type through the registry instead of expanding a closed OpenAPI enum.



Type-specific richness

Profiles can evolve compute, storage, service, and future domains without polluting the common Resource.



Discoverable topology

Named relations expose real facility relationships without encoding them in identifiers or URL patterns.



Portable clients

Clients rely on advertised semantics instead of facility-specific route construction and SDK conventions.



Safer agents

Agents can ground decisions in live links and governing profiles instead of speculating about API paths.



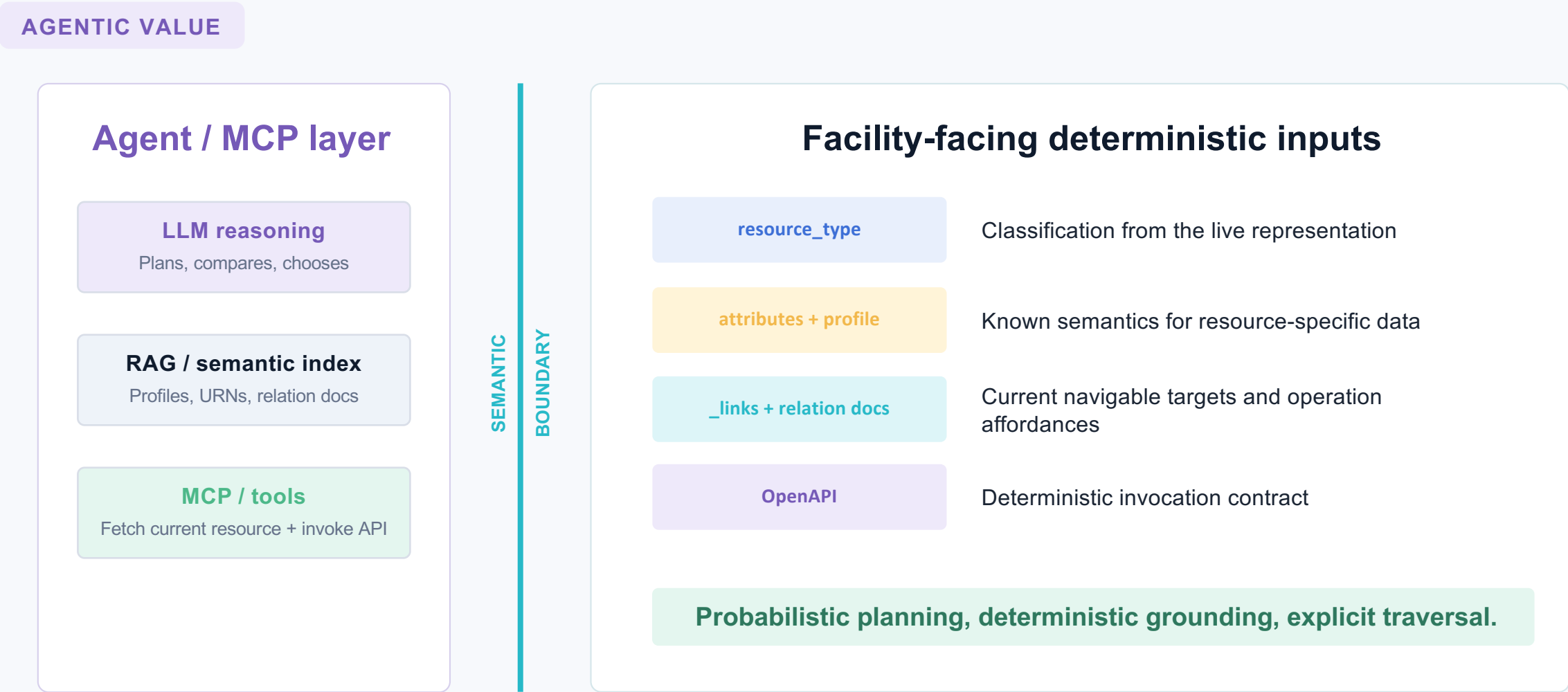
Graceful evolution

Unknown types, profiles, or relations can be handled explicitly instead of breaking the entire common schema.

Derived from the current registry model, type-specific attributes RFC, HAL RFC, and Resource profile

Why this matters for agentic workflows

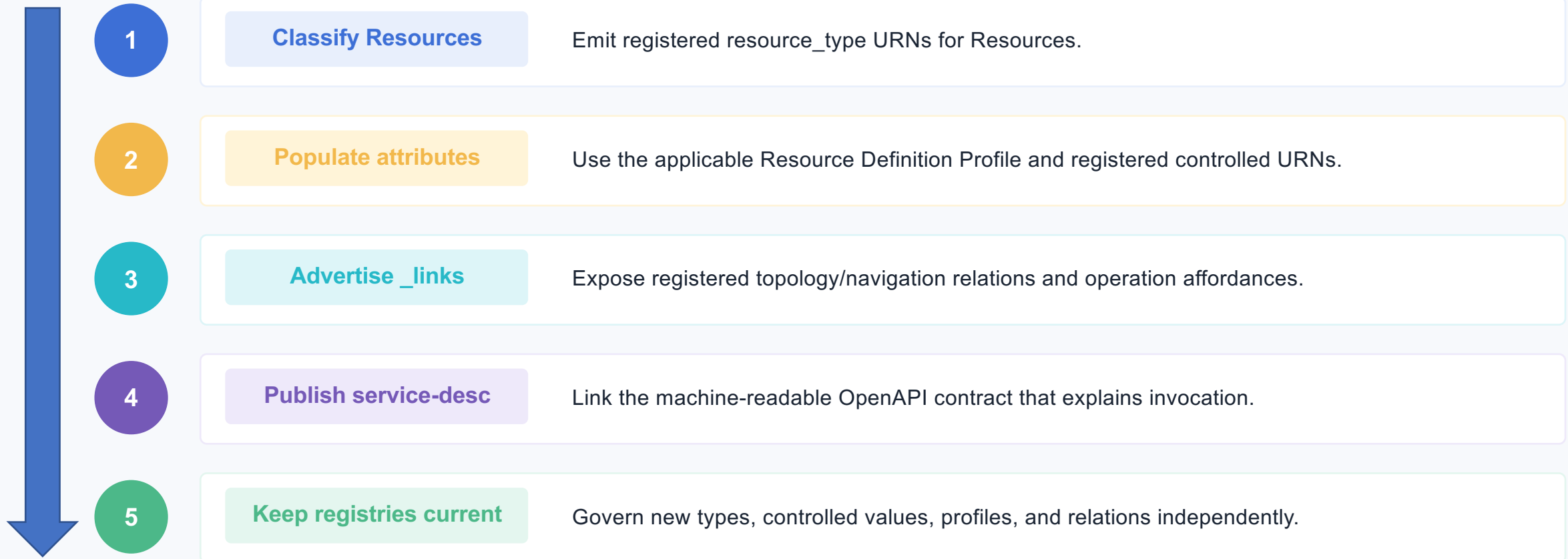
The architecture creates a deterministic semantic boundary between probabilistic reasoning and facility APIs.



What a facility needs to publish

The model is incremental: facilities can adopt the semantic layers without redesigning every API at once.

ADOPTION



No client should need to infer a facility's URL structure to understand or navigate its advertised IRI capabilities.

Sources: [registry/README.md](#); [registry/urns/*.md](#); [registry/profiles/*](#); [registry/relations/*](#); [rfc/rfc-hal-links.md](#)

Takeaways

A self-describing Resource model gives humans, software clients, and agents the same semantic contract.

SUMMARY

resource_type

Classifies what the Resource is using a governed URN taxonomy.

attributes + profile

Carries type-specific data whose semantics evolve independently of the common schema.

_links + relation docs

Makes topology and applicable operations explicit and traversable.

OpenAPI

Remains the structural and invocation contract.

Architectural objective: make facility capabilities discoverable enough that an agent can reason about them without inventing semantics or paths.

Primary repository references

registry/README.md • registry/urns/resource-types.md • registry/urns/attributes.md • registry/profiles/status/resource.md • rfc/rfc-type-specific-attributes.md • rfc/rfc-hal-links.md • registry/relations/README.md